LLM_project Public

Watch 0 · Fork 1 · Star 0

main · 3 Branches · 0 Tags · Go to file · t · Add file · Code

mihoyoMhb Change a few typos · aa176a8 · 3 weeks ago · 21 Commits

data	Change a few typos	3 weeks ago
experiment_results	Updated memory evaluation analysis	last month
memory	Update documentation and add entity-aware memory mode.	last month
rag_modules	Update cross-platform support	last month
.gitignore	Update LLM Project, with memory database and rolling me...	last month
DOCUMENTATION_UPDATE.md	Update documentation and add entity-aware memory mode.	last month
LICENSE	Updated the LICENSE file.	last month
README.md	Updated the README.md file.	last month
print_memory.py	Update LLM Project, with memory database and rolling me...	last month
requirements.txt	Updated README.md and environment.yml	last month
run_gui.py	Update documentation and add entity-aware memory mode.	last month
setup_config.py	Update cross-platform support	last month
streamlit_app.py	Change a few typos	3 weeks ago
test_entity_aware_memory.py	Update documentation and add entity-aware memory mode.	last month
test_memory_integration.py	Update LLM Project, with full functionality memory database...	last month

About

No description, website, or topics provided.

- Readme
- MIT license
- Activity
- 0 stars
- 0 watching
- 1 fork

Report repository

Releases

No releases published
Create a new release

Packages

No packages published
Publish your first package

Contributors 2

mihoyoMhb
moutushisen Moutushi Debnath Sen

Languages

Python 100.0%

Suggested workflows
Based on your tech stack

Python package · Configure
Create and test a Python package on multiple Python versions.

Python application · Configure
Create and test a Python application.

Django · Configure
Build and Test a Django Project

More workflows · Dismiss suggestions

README · MIT license

Study Pal - AI Document Assistant

> Intelligent RAG-powered assistant with Entity-Aware Memory
> Remembers key concepts and technical terms—perfect for learning!

python 3.9 | 3.10 | 3.11 | 3.12 · license MIT

Tech Stack: LangChain • Google Gemini • Ollama • Streamlit • FAISS

📑 Table of Contents

1. ⚡ Quick Start
2. 🧠 Memory Modes
3. 🏗️ Core Features
4. 🚀 Installation
5. 💡 Usage Guide
6. 🛠️ Advanced Configuration
7. 🐛 Troubleshooting
8. 📚 Project Architecture
9. ❓ FAQ

⚡ Quick Start

Command	What it does
`python run_gui.py`	Start with **standard memory**
`python run_gui.py -e`	Start with **entity-aware memory** (recommended for learning)
`python print_memory.py`	View current memory
`python setup_config.py`	Configure API keys

🧠 Memory Modes: Standard vs Entity-Aware

📊 Comparison

Metric	Standard	Entity-Aware (`-e`)
Entity Preservation	37%	65% 🚀 (+27%)
Key-Point Recall	66%	79% 🚀 (+12%)
Compression Ratio	17%	21%
Best for	Casual chat	Learning & tech

Based on 505-sample evaluation with 95% confidence intervals

💡 Examples

Learning Python inheritance:

```
User: "What is inheritance in Python? Use class Child(Parent). What's MRO?"
```

Standard Memory:

> ❌ "Discussed Python class concepts and method resolution"
>
> - Lost: `Child(Parent)`, `MRO`

Entity-Aware Memory:

> ✅ "Python inheritance uses `class Child(Parent)` syntax. MRO (Method Resolution Order) determines method priority"
>
> - Preserved: All key terms!

🎓 When to use each mode

Use Entity-Aware (`-e`):

- 📚 Learning programming/frameworks
- 🔬 Technical documentation
- 🧮 Math/algorithms with formulas
- 💻 Code syntax/commands

Use Standard (default):

- 💬 Casual conversations
- 🎨 Creative writing
- 🤔 General Q&A

🏗️ Core Features

- 📄 **Smart PDF Q&A** - Upload documents, ask questions, get cited answers
- 🧠 **Entity-Aware Memory** - Preserves technical terms, code, concepts (65% → 85%)
- 🤖 **Dual Models** - Google Gemini API or local Ollama
- 🔒 **Privacy-First** - All data stored locally in SQLite
- 🎨 **Modern UI** - Split-screen PDF viewer + chat

🚀 Installation

Quick Install (5 steps)

```
# 1. Create environment
conda create -n rag_system python=3.12 -y
conda activate rag_system

# 2. Install dependencies (5-10 min)
pip3 install torch torchvision
pip install -r requirements.txt

# 3. (Optional) Install local model
curl -fsSL https://ollama.ai/install.sh | sh  # Linux/WSL
brew install ollama                           # macOS
ollama pull phi3:mini

# 4. Configure API key
python setup_config.py
# Get key: https://makersuite.google.com/app/apikey

# 5. Launch!
python run_gui.py -e  # Entity-aware mode (recommended)
# Access: http://localhost:8501
```

▶ 📦 **Don't have Conda? Install Miniconda first**

System Requirements

Component	Requirement
Python	3.9 - 3.12
RAM	8GB min, 16GB+ recommended
Storage	~5GB
OS	Linux, macOS, Windows (WSL2)

💡 Usage Guide

1. Basic Usage

```
# Launch app
python run_gui.py -e

# Access: http://localhost:8501
```

In the web interface:

1. 📄 **Upload PDF** → Load document
2. 💬 **Ask questions** → Type in chat
3. 📚 **View sources** → Expand "Reference Sources"
4. 🤖 **Switch models** → Toggle Gemini/Ollama in sidebar

2. Memory System

Create personalized memory:

```
You: "I'm a beginner programmer, explain things simply"
AI: [responds with simple explanation]
→ Click "✨ Generate & Merge" in sidebar
→ AI remembers this for future sessions!
```

Example profiles:

Profile	Memory Input	Result
🎓 Beginner	"I'm learning Python, need simple explanations"	Uses analogies, avoids jargon
🔬 Researcher	"I'm an ML researcher, provide technical depth"	Includes math, citations
📄 Student	"Preparing for exams, focus on key concepts"	Structured summaries

Useful commands:

- `python print_memory.py` - View current memory
- Click "Clear Memory" button - Reset all memories

🛠️ Advanced Configuration

🔧 Conda Environment Management

▶ **View environment commands**

🤖 Additional Ollama Models

```
ollama list                    # View installed
ollama pull mistral:7b         # Mistral (balanced)
ollama pull codellama:7b       # Code-specialized
ollama pull qwen2.5:7b         # Better entity preservation
```

💻 Programmatic API

```
# Use entity-aware memory in your code
from memory.entity_aware_generator import (
    generate_entity_aware_memory,
    extract_key_terms
)

# Generate memory
memory = generate_entity_aware_memory(
    chat_pairs=[("Q", "A"), ...],
    history_text="...",
    max_chars=1200,
    model_name="gemini-2.5-flash-lite",
    verbose=True  # Show stats
)

# Extract entities only
entities = extract_key_terms("Learning Python classes")
# Output: {'Python', 'classes'}
```

🎮 GPU Acceleration (Optional)

```
# Check CUDA
python -c "import torch; print(torch.cuda.is_available())"

# Install PyTorch with CUDA
pip install torch torchvision --index-url https://download.pytorch.org/whl/cu121
```

🐛 Troubleshooting

Problem	Solution
❌ **ModuleNotFoundError**	`conda activate rag_system`
❌ **Ollama connection failed**	`ollama serve`
❌ **Memory not saving**	Check `ls data/memory.db`, run `python print_memory.py`
❌ **API key error**	`python setup_config.py` to reconfigure
❌ **Import errors**	`pip install -r requirements.txt --force-reinstall`
❌ **WSL can't access**	Get WSL IP: `ip addr show eth0`, access `http://[WSL_IP]:8501`

▶ **Entity-Aware mode not working?**

📚 Project Architecture

📁 File Structure

```
LLM_project/
├── memory/                          # Memory system core
│   ├── generator.py                 # Standard memory
│   ├── entity_aware_generator.py    # Entity-aware (85% retention)
│   └── rolling.py                   # SQLite storage
├── rag_modules/                     # RAG implementation
│   ├── providers/                   # Gemini, Ollama providers
│   ├── core/                        # RAG chain builder
│   └── utils/                       # PDF utilities
├── streamlit_app.py                 # Web UI
├── run_gui.py                       # Launcher with -e flag support
└── test_entity_aware_memory.py      # Memory comparison tool
```

🔍 Testing Scripts

```
python test_entity_aware_memory.py    # Compare memory modes
python test_memory_integration.py     # Integration tests
python print_memory.py                # View stored memory
```

📖 Documentation

- Entity-Aware Memory Guide - Detailed usage
- Cognee Inspiration - Design philosophy

❓ FAQ

▶ **Which memory mode should I use?**

▶ **Can I switch modes during a session?**

▶ **Does entity-aware mode slow down the app?**

▶ **How is this different from Cognee knowledge graphs?**

▶ **Where is data stored?**

▶ **Can I customize entity detection?**

🔐 Security & Privacy

- 🔒 API keys in `~/.config/llm_project/.env` (never committed)
- 🛡️ All data stored locally (no cloud uploads)
- 🗑️ Clear memory anytime with "Clear Memory" button

📝 License

MIT License - See LICENSE for details.

Copyright (c) 2025 mihoyohb, group 5, LLM Course 1RT730, Uppsala University, Sweden

---

🎓 Changelog

v1.1.0 (Current) - Entity-Aware Memory Release

Key Improvements:

- 🧠 **Entity-Aware mode** (`-e` flag): 37% → 65% entity preservation (+27%)
- 📊 **Key-point recall**: 66% → 79% (+12%)
- ⚡ **Minimal overhead**: +1s generation time, same compression ratio

New Files:

- `memory/entity_aware_generator.py` - Entity-aware generator
- `test_entity_aware_memory.py` - Comparison tool

Inspired by: Cognee Knowledge Graph System

v1.0.0 - Initial Release

- RAG-powered PDF Q&A
- Standard memory system
- Gemini + Ollama support

🔗 Resources

Resource	Link
LangChain	https://python.langchain.com/
Ollama Models	https://ollama.ai/library
Google Gemini	https://ai.google.dev/
Cognee (Inspiration)	https://github.com/topoteretes/cognee

Version: 1.1.0 | **Last Updated:** 2025 | **Maintained by:** Study Pal Team (Group 5, Uppsala University)